

Break-It Guide — finding & proving bugs

The companion to the Build-It Flow handout, for the Break-It phase. You've built your `stor` ; now you attack every other qualifying team's. This is the participant reference for what you attack, how breaks are judged, how to write one, and the pitfalls to avoid. For point values and ranking see 04-scoring.md; for the `stor` spec see 02-problem-statement.md.

1. The goal & your break package

For each qualifying target team you submit **machine-checkable test cases** that prove a bug in their `stor` . The organizers publish a **break package** per target (download it from the dashboard). Open it, read the source, find a flaw, and prove it.

You get (in the break package)	You do not get (kept on our server)
The team's full source & a prebuilt binary for local testing	The users' secret key (token)
The standard <code>enc.db</code> (see §2)	The secret file contents stored inside it
A README — the target's numeric team ID , categories & scoring	

Your job: leverage or tamper these hidden values — without the key. A correct `stor` keeps them safe; a buggy one lets you recover or forge them anyway.

The grader recompiles the target's source from their repo before testing. Your exploit must work against the **freshly built** binary — not just the prebuilt copy in the package.

2. How your target's `enc.db` is built (the attack surface)

Before Break-It opens, the organizers build **one** `enc.db` **per team** — **by running that team's own** `stor` . So its format and encryption are exactly what *they* implemented (strong or weak).

First — three random secrets are generated and kept on our server (never shared):

Secret	Role
the key (token)	the shared key used by both users
file1 's content	the integrity target
file2 's content	the confidentiality target

Then — the team's `stor` is run with those secrets, writing the database:

```
stor -u user1 -k <key> register
stor -u user1 -f file1 create
stor -u user1 -k <key> -f file1 write "<file1's content>"      # the integrity target
stor -u user2 -k <key> register
stor -u user2 -f file2 create
stor -u user2 -k <key> -f file2 write "<file2's content>"      # the confidentiality targ
```

The resulting `enc.db` ships to you; the key and the two contents stay on the server. **If the team stored `enc.db` in plaintext, the secrets are sitting in the open** — that itself is the bug you exploit for confidentiality. A properly encrypted `enc.db` makes them unreadable.

Targets are fixed. Integrity is always judged on `user1/file1` and confidentiality on `user2/file2` — those are the only files the grader checks for each type, no matter what your commands touch.

3. The five break types — and how each is judged

Every break declares one `type`. Here is each value, its points, and **exactly how the grader decides it's valid**:

<code>type</code>	Points	What it proves	Grader's success condition
<code>correctness</code>	25	Wrong output for valid input	Target's output differs from a trusted reference <code>stor</code> (the "oracle" — see §4)
<code>crash</code>	50	Memory-safety break / DoS	A command's process is killed by a signal (e.g. SIGSEGV), or runs longer than 60 s
<code>integrity</code>	100	Modify a victim's file	Re-reading <code>user1/file1</code> with the real key returns content different from the original
<code>confidentiality</code>	100	Recover a victim's secret	Your declared <code>output</code> matches <code>user2/file2</code> 's real content
<code>security</code>	100	Hijack control-flow to <code>win()</code>	The exact marker Arbitrary access achieved! appears in the command output

Integrity, confidentiality and security are **three distinct** `type`s but all score **100** — they each defeat the security model. There is **no penalty** for a failed attempt (but each counts as a submission). "Availability/DoS" is **not** a separate type — a hang or timeout is filed under **crash**.

Scoring at a glance (full model in **04-scoring.md**): every point you take is a point the builder loses. The **live scoreboard is provisional** — it shows every valid break at full value. In the **final standings** a bug found by several teams counts **once** and its points are **split** among the finders — so a bug **nobody else finds** keeps its full value. Prioritise **unique** and **high-severity (100-point)** bugs.

4. How `correctness` is judged: the reference oracle

Correctness has no secret to recover, so how does the grader know the target is *wrong* and not just different from what you claimed? It runs your commands **twice**:

1. on a trusted **reference** `stor` (the oracle — a known-correct implementation), and
2. on the **target's** `stor`.

The break is **valid only if the target's output differs from the oracle's**. If you supply an `expected` field, it **must agree with the oracle** — claiming a wrong "correct" answer gets the break **rejected** (this keeps reports honest). So craft a case where you genuinely know the right answer and the target gets it wrong. The published **functionality test suite is the authoritative definition** of correct behavior; the oracle is validated against it.

5. Where & how to submit

Exactly like Build-It: **push to your team's own repository**. Each break is a folder under `break/` with **two required files**:

```
break/<your-test-name>/
├─ test.json      # required: the machine-checkable test
└─ description.txt # required: plain-English explanation of the bug + how the test tr
```

Push → the contest picks it up automatically → results appear on your **dashboard** (valid / invalid).
The flow per push:

1. You push `break/<name>/test.json`.
2. We **rebuild the target team's** code and run your test against it.
3. The result (valid or not) appears on your dashboard.

6. Anatomy of a `test.json`

```
{
  "target_team": 12,
  "type": "security",
  "commands": [
    {"program": "stor", "args": ["-u", "alice", "-k", "s3", "register"]},
    {"program": "stor", "args": ["-u", "alice", "-f", "x", "-k", "s3", "write"],
     "stdin": "AAAA...payload"}
  ]
}
```

Field	Meaning
<code>target_team</code>	The numeric team ID of your target (from the dashboard / README).
<code>type</code>	One of <code>correctness</code> , <code>crash</code> , <code>integrity</code> , <code>confidentiality</code> , <code>security</code> .
<code>commands</code>	The ordered sequence of <code>stor</code> calls to run — each is <code>program</code> + <code>args</code> .
<code>commands[].stdin</code>	(<i>optional</i>) a string piped to that command's standard input.
<code>commands[].output</code>	(<i>confidentiality</i>) the secret value you claim to have recovered — the grader compares it to the real content.
<code>expected</code>	(<i>correctness</i>) the spec-correct result per command; must match the oracle .

7. Worked examples

Illustrative bug classes — the kind of flaw each `type` targets, with the matching `test.json`.

Correctness (25) — substring match returns the wrong file

A `stor` that locates a file with `strstr` (substring) instead of an exact compare: searching for file `"a"` also matches the entry for `"ab"`.

```
{ "target_team": 12, "type": "correctness",
  "commands": [
    {"program": "stor", "args": ["-u", "u", "-f", "ab", "-k", "k", "write", "FIRST"]},
    {"program": "stor", "args": ["-u", "u", "-f", "a", "-k", "k", "write", "SECOND"]},
    {"program": "stor", "args": ["-u", "u", "-f", "a", "-k", "k", "read"]} ],
  "expected": [ {"exit": 0}, {"exit": 0}, {"stdout": "SECOND", "exit": 0} ] }
```

Reading `a` should print `SECOND` (what the oracle prints). A buggy `stor` matches `ab` first and prints `FIRST` → the outputs differ → valid correctness break.

Crash (50) — overflow a fixed buffer

A `stor` that copies attacker input into a fixed stack buffer with no bounds check (`char buf[64]; strcpy(buf, input);`). A long write runs off the end and crashes.

```
{ "target_team": 12, "type": "crash",
  "commands": [
    {"program": "stor", "args": ["-u", "a", "-k", "s", "register"]},
    {"program": "stor", "args": ["-u", "a", "-f", "big", "create"]},
    {"program": "stor", "args": ["-u", "a", "-f", "big", "-k", "s", "write"],
      "stdin": "AAAAAAAA...(a few thousand A's)...AAAA" } ] }
```

Success = the process is killed by a signal **or** any command runs past 60 s (a hang counts too).

Confidentiality (100) — plaintext `enc.db` leaks the secret

If the team writes data unencrypted, `user2/file2`'s content is readable straight out of `enc.db`. Put the value you recovered in `output`:

```
{ "target_team": 12, "type": "confidentiality",
  "commands": [
    {"program": "stor", "args": ["-u", "user2", "-f", "file2", "read"],
      "output": "the_secret_you_recovered" } ] }
```

The grader reads `user2/file2` with the real key and checks your `output` matches.

Integrity (100) — forge a record

If filenames are written raw into a line-based `enc.db`, a newline in a filename injects a forged line — overwriting `user1/file1` without the key:

```
{ "target_team": 12, "type": "integrity",  
  "commands": [  
    {"program": "stor", "args": ["-u", "att", "-k", "k", "register"]},  
    {"program": "stor", "args": ["-u", "att", "create", "-f",  
      "x\nFILE file1 USER user1 CONTENTS HACKED"]} ] }
```

No `output` needed — just corrupt it. The grader re-reads `user1/file1` with the real key; if the content changed, the break is valid.

Security / control-flow (100) — reach `win()`

Every `stor` contains a `win()` that prints `Arbitrary access achieved!` but is never called normally. The mandated build flags (`-m32 -fno-stack-protector` + executable stack) make a classic stack smash feasible: overflow a buffer and overwrite the saved return address with `win`'s. The grader's only check is that the marker string appears in the output. *(Crafting the exact payload is real work — the address depends on the target's binary, which you have for local testing.)*

8. Key gotchas

- **integrity counts only on user1/file1** — the one file the grader re-reads for tampering.
- **confidentiality counts only on user2/file2** — the only secret the grader checks you recovered.
- **One folder per break** — each `break/<name>/` is exactly **one target + one category**. To hit several teams or several categories, make several folders.
- **Both files required** — `test.json` **and** `description.txt`, or the break is rejected.
- **Only a changed test.json registers** — a commit touching only `description.txt` won't; re-push the `test.json` to iterate (the new one replaces your previous one of that name).
- **We recompile the target** — beat the freshly built binary. Each command is killed after **60 s** (a timeout counts as a crash); test input files must not exceed **600 MB**.
- **To count as valid:** confidentiality needs your recovered secret in `output`; security needs the exact `Arbitrary access achieved!` marker; correctness' `expected` must match the reference oracle.

Rules: don't attack the contest infrastructure itself — only other teams' `stor`; each team does its own work; share how you broke another team with them only after the relevant deadline. Organizers' decisions are final. Points, splitting and ranking: see 04-scoring.md.